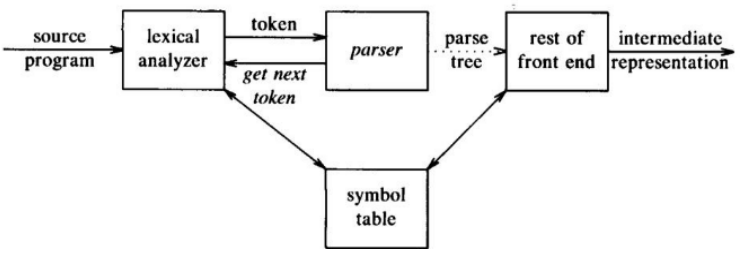
SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution and affiliated to Anna University, Chennai)
Kuniamuthur, Coimbatore - 641 008

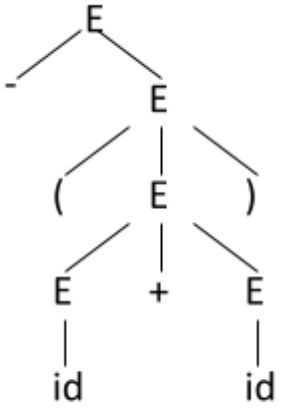
DEPARTMENT OF INFORMATION TECHNOLOGY

23CSC04-PRINCIPLES OF COMPILER DESIGN

QUESTION BANK MODULE - I

PART-A

1.	List the specification of tokens. **Answer:** There are 3 specifications of tokens: 1)Strings 2) Language 3)Regular expression
2.	Recall the basic three parts of YACC implementation. **Answer:** /* definitions */ .... %% /* rules */ .... %% /* auxiliary routines */
3.	Comment on Parser and its roles. **Answer:** The parser obtains a string of tokens from the lexical analyzer and verifies that the string can be the grammar for the source language. It detects and reports any syntax errors and produces a parse tree from which intermediate code can be generated. 
4.	Infer the syntax tree for the expression –(id+id). **Answer:** **E → E + E | E – E | E * E | E / E | - E** **E → (E)** **E → id**



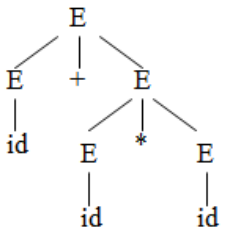
5. Identify the given grammar E->E+E;E->E*E;E->(E);E->id, is ambiguous or not.

Answer:

Assume the String id+id*id

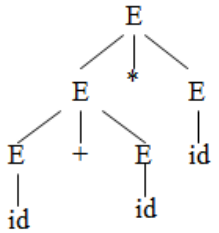
Left Most Derivation 1:

$$E \Rightarrow E+E \Rightarrow id+E \Rightarrow id+E*E$$
$$\Rightarrow id+id*E \Rightarrow id+id*id$$



Left Most Derivation 2:

$$E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow id+E*E$$
$$\Rightarrow id+id*E \Rightarrow id+id*id$$



6. Describe Symbol Table and Construct the symbol table for a simple program of adding two numbers.

Answer:

A symbol table is a data structure containing a record for each identifier, with fields for the attributes of the identifier. These attributes provide information about the storage allocated for an identifier, its type, its scope and for procedure names, things such as number, types of its arguments, method of passing each argument and the type returned.

Example:

```
int add(int a, int b)
{
int sum = 0;
sum = a + b;
return sum;
}
```

Symbol Table:

Name	Type	Scope
add	function	Global
a	int	function parameter
b	int	function parameter
sum	int	Local

7. Identify the difference between LL and LR Parser.

Answer:

LL Parser	LR Parser

First L of LL is for left to right and second L is for leftmost derivation.	L of LR is for left to right and R is for rightmost derivation.
It follows the left most derivation.	It follows reverse of right most derivation.
Using LL parser parser tree is constructed in top down manner.	Parser tree is constructed in bottom up manner.
In LL parser, non-terminals are expanded.	In LR parser, terminals are compressed.
Starts with the start symbol(S).	Ends with start symbol(S).
Ends when stack used becomes empty.	Starts with an empty stack.
Pre-order traversal of the parse tree.	Post-order traversal of the parser tree.
Terminal is read after popping out of stack.	Terminal is read before pushing into the stack.
It may use backtracking or dynamic programming.	It uses dynamic programming.
LL is easier to write.	LR is difficult to write.
Example: LL(0), LL(1)	**Example:** LR(0), SLR(1), LALR(1), CLR(1)

8. Define sentinel, buffer pairs.

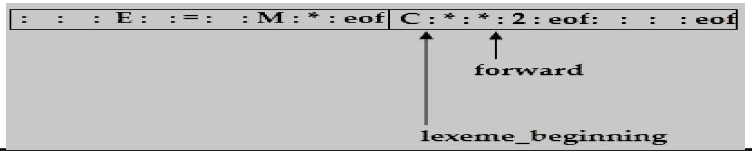
Answer:

Buffer Pairs:

- Forward pointer, scans ahead until a match for a pattern is found.
- Once the next lexeme is determined, the forward pointer is set to the character at its right end.
- If the forward pointer is about to move past the halfway mark, the right half is filled with N new input characters.
- If the forward pointer is about to move past the right end of the buffer, the left half is filled with N new characters and the forward pointer wraps around to the beginning of the buffer.

Sentinals:

- In the Buffer pairs scheme, every time the forward pointer is moved, it checks in order to ensure that one half of the buffer did not move off. If it is done, then the other half should be reloaded.
- Hence, the ends of the buffer halves need to go through two tests for every advance of the forward pointer.
- Test 1: For the end of the buffer.
- Test 2: To determine what character is to be read.
- The usage of sentinel helps to reduce the two tests that are required into one by extending each buffer half to hold a sentinel character at the end. (eof character is used as sentinel).

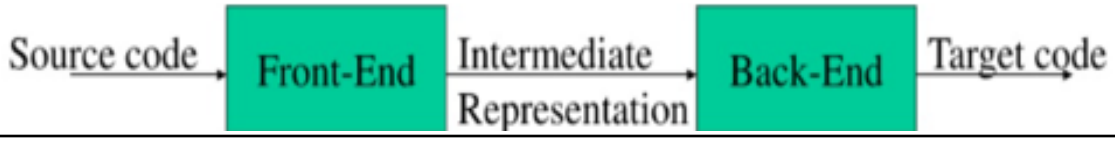
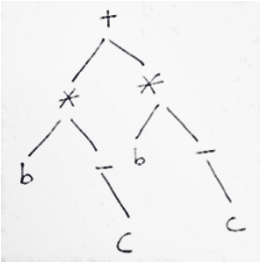


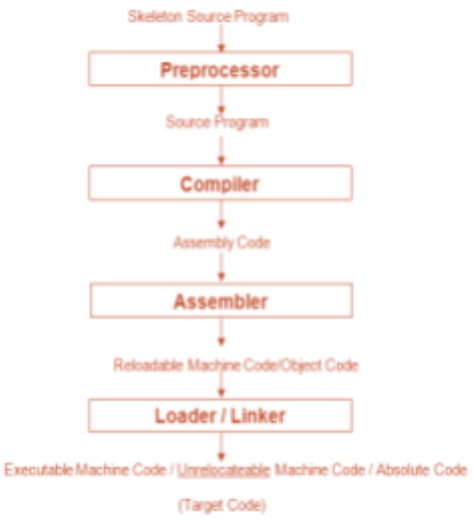
9. Build a grammar from eliminating left recursion in the following production:
S->S*C/C; C->Cb/F; F->id

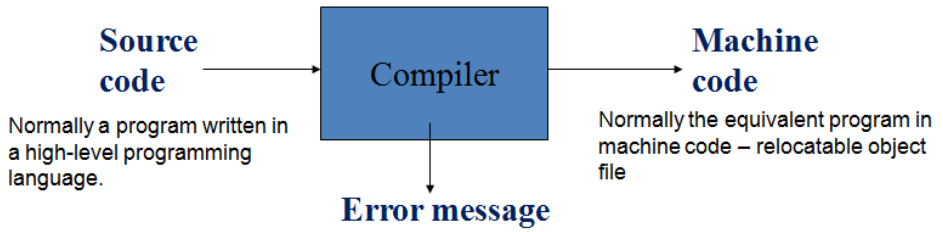
Answer:

Eliminating Left Recursion:

S->S*C/C

	S->CS` S`□*CS`/ ε  **C->Cb/F** C□FC` C`□bC`/ε **Grammer after Eliminating Left Recursion:** S->CS` S`□*CS`/ ε C□FC` C`□bC`/ε F->id
10.	Recall the basic two parts of the compilation process. **Answer:** There are two parts to compilation: ● In analysis part, an intermediate representation is created from the given source program. ▪ Lexical Analyzer, Syntax Analyzer and Semantic Analyzer are present in this part ● In synthesis part, the equivalent target program is created from this intermediate representation. ▪ Intermediate Code Generator, Code Generator, and Code Optimizer are present in this part. 
11.	Construct the syntax tree for the expression a=b*-c+b*-c. **Answer:** 

12.	Illustrate dramatically how a language is processed. **Answer:** 
13.	Eliminate left recursion from the following grammar. A->Ac/Aad/bd **Answer:** A → bdA' / A' A' → cA' / adA' / ∈
14.	Derive the string aaabbabbba using the grammar below(using leftmost derivation and rightmost derivation). S->aB|bA A->a|aS|bAA B->b|bS|aBB **Answer:** **<u>Left Most Derivation:</u>** S → a**B** → aa**B**B (Using B →aBB) → aaa**B**BB (Using B → aBB) → aaab**B**B (Using B → b) → aaabb**B** (Using B → b) → aaabba**B**B (Using B → aBB) → aaabbab**B** (Using B → b) → aaabbabb**S** (Using B → bS) → aaabbabbb**A** (Using S → bA) → aaabbabbba (Using A → a) **<u>Rightmost Derivation:</u>** S → a**B** → aaB**B** (Using B → aBB) → aaBaB**B** (Using B → aBB) → aaBaBb**S** (Using B → bS) → aaBaBbb**A** (Using S → bA) → aaBa**B**bba (Using A → a) → aa**B**abbba (Using B → b) → aaaB**B**abbba (Using B → aBB) → aaa**B**babbba (Using B → b) → aaabbabbba (Using B → b)

15.	Draw NFA and transition table for the regular expression (ab* | ba*) **Answer:** ****Refer Thompson's construction and find solution.**
16.	Define tokens, patterns and lexemes. **Answer:** **Token:** Token is an abstract symbol representing a kind of lexical unit. ie, a particualr keyword, an operator, constants, literal strings, etc.. A token is a pair consisting of a token name and an optional attribute value. **<token name, attribute value>** **Example:** rate is identifier +,-,/,* are operator if, then, go are keyword **Pattern:** A pattern is a description of the form that the lexemes of a token may take. **Example:** If the token is a keyword, the pattern is the sequence of characters that form the keyword. If the token is an identifier, the pattern is more complex structure that is matched by many strings. **Lexeme:** A lexeme is a sequence of characters in the source program that matches the pattern for a token and is identified by the lexical analyzer as an instance of that token.
17.	Write the algorithm for left factoring of a grammar. **Answer:** • In general, $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ where α is non-empty and the first symbols of β_1 and β_2 (if they have one)are different. • when processing α we cannot know whether expand A to $\alpha\beta_1$ or A to $\alpha\beta_2$ • But, if we re-write the grammar as follows $A \rightarrow \alpha A'$ $A' \rightarrow \beta_1 \mid \beta_2$ so, we can immediately expand A to $\alpha A'$
18.	Identify the significance of compiler with an example. **Answer:** A **compiler** is a program takes a program written in a source language and translates it into an equivalent program in a target language.  **Example:** C Compiler
19.	Enlist cousins of the compiler and give its importance. **Answer:** 1. Preprocessor 2. Assembler 3. Loader and Link-editor In a language processing system, the source code is first preprocessed. The modified source program is processed by the compiler to form the target assembly program which is

then translated by the assembler to create relocatable object codes that are processed by linker and loader to create the target program.

20. Illustrate why buffering is used in Lexical Analysis? List out the commonly used buffering methods.

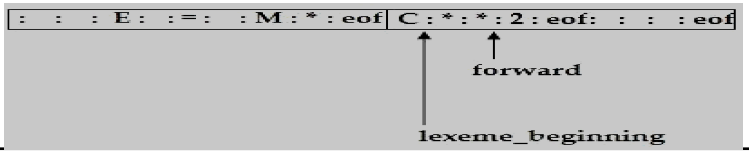
Answer:

Buffer Pairs:

- Forward pointer, scans ahead until a match for a pattern is found.
- Once the next lexeme is determined, the forward pointer is set to the character at its right end.
- If the forward pointer is about to move past the halfway mark, the right half is filled with N new input characters.
- If the forward pointer is about to move past the right end of the buffer, the left half is filled with N new characters and the forward pointer wraps around to the beginning of the buffer.

Sentinals:

- In the Buffer pairs scheme, every time the forward pointer is moved, it checks in order to ensure that one half of the buffer did not move off. If it is done, then the other half should be reloaded.
- Hence, the ends of the buffer halves need to go through two tests for every advance of the forward pointer.
- Test 1: For the end of the buffer.
- Test 2: To determine what character is to be read.
- The usage of sentinel helps to reduce the two tests that are required into one by extending each buffer half to hold a sentinel character at the end. (eof character is used as sentinel).



21. Compare tokens, patterns and lexemes with examples.

Answer:

Criteria	Token	Lexeme	Pattern
Definition	Token is basically a sequence of characters that are treated as a unit as it cannot be further broken down.	It is a sequence of characters in the source code that are matched by given predefined language rules for every lexeme to be specified as a valid token.	It specifies a set of rules that a scanner follows to create a token.
Interpretation of type Keyword	all the reserved keywords of that language(main, printf, etc.)	int, goto	The sequence of characters that make the keyword.

Interpretation of type Identifier	name of a variable, function, etc	main, a	it must start with the alphabet, followed by the alphabet or a digit.
Interpretation of type Operator	all the operators are considered tokens.	+, =	+, =
Interpretation of type Punctuation	each kind of punctuation is considered a token. e.g. semicolon, bracket, comma, etc.	(,), {, }	(,), {, }
Interpretation of type Literal	a grammar rule or boolean literal.	“Welcome to GeeksforGeeks!”	any string of characters (except ‘ ‘) between ” and “

22. Construct the Regular expression for the set of input alphabets a & b in which set of strings have zero or any number of a and b followed by bba.
Answer:
Regular Expression:
(ab)*bba

23. Compute regular definition for an identifier.
Answer:
To write regular expression for some languages can be difficult, because their regular expressions can be quite complex. In those cases, we may use *regular definitions*.
A ***regular definition*** is a sequence of the definitions of the form:
d1 -> r1
d2 -> r2
…
dn -> rn
Example:
letter_ -> A | B | … | Z | a | b | … | Z | _
digit -> 0 | 1 | … | 9
id -> letter_ (letter_ | digit)*

24. Paraphrase the use of sentinels. Give an example for Left Most Derivation using sentinels.
Answer:
Given a grammar G with start symbol S, if S → α, where α may contain non-terminals or terminals, then α is called the sentinel form of G.
Example: Given grammar G : E → E+E | E*E | (E) | - E | id
The sentence id+id*id has the following leftmost derivation:
E → E+ E
E → E* E
E → id + E
E→E+ E * E
E → id + E * E
E → id + E * E
E → id + id * E
E → id + id * E

E → id + id * id
E → id + id * id

25. Differentiate bottom up and top-down parser with an example.

Answer:

Top Down Parser	**Bottom Up Parser**
Parse Tree can be built from root to leaves.	Parse tree can be built from leaves to root.
Simple to implement	Complex to Implement
Less efficient parsing techniques. Problems like ambiguity and left recursion occur.	More efficient parsing techniques. Efficiently handles conflicts occurring parsing table
Applicable to small class of languages	Applicable to broad class of languages
Ex: 1) Recursive Descent Parser 2) Predictive Parser	Ex: 1) Shift Reduce Parser 2) LR Parser

26. Differentiate Compiler and Interpreter.

Answer:

Parameter	Compiler	Interpreter
Machine Code	It stores the machine language on the disk in the form of machine code.	It doesn't save the machine language at all.
Running Time	The compiled codes run comparatively faster.	The interpreted codes run comparatively slower.
Model	It works on the basis of the language-translation linking-loading model.	It works on the basis of the Interpretation method.
Generation of Program	It generates an output program in the exe format. A user can run it independently from the originally intended program.	It doesn't generate an output program. Meaning, it evaluates the source program every time during individual execution.
Machine Code	It stores the machine language on the disk in the form of machine code.	It doesn't save the machine language at all.
Running Time	The compiled codes run comparatively faster.	The interpreted codes run comparatively slower.

27. List out compiler construction tools with its significance.

Answer:

These are specialized tools that have been developed for helping implement various phases of a compiler. The following are the compiler construction tools:

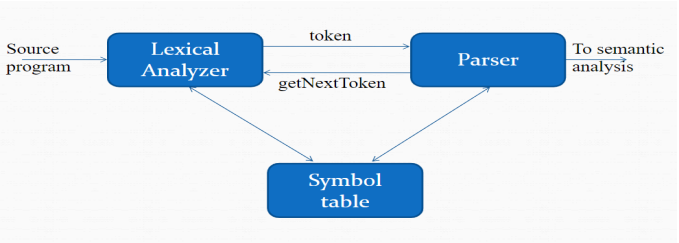
1) Parser Generators

2) Scanner Generator
3) Syntax-Directed Translation
4) Automatic Code Generators
5) Data-Flow Engines

28. Illustrate the role of a lexical analyzer.

Answer:

The role of Lexical Analyzer in compiler design is to read character streams from the source code, check for legal tokens and pass the data to the syntax analyzer when it demands.



29. Discuss Handle Pruning with an example.

Answer:

A right-most derivation in reverse can be obtained by **handle-pruning**.

$S=\gamma_0 \Rightarrow \gamma_1 \Rightarrow \gamma_2 \Rightarrow \ldots \Rightarrow \gamma_{n-1} \Rightarrow \gamma_n = \omega$

ω-input string

Start from γ_n, find a handle $A_n \rightarrow \beta_n$ in γ_n, and replace β_n in by A_n to get γ_{n-1}.

Then find a handle $A_{n-1} \rightarrow \beta_{n-1}$ in γ_{n-1}, and replace β_{n-1} in by A_{n-1} to get γ_{n-2}.

Repeat this, until we reach S.

Example:

E → E+T|T
T → T*F|F
F → (E)|id

RIGHT SENTENTIAL FORM	HANDLE	REDUCING PRODUCTION
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \rightarrow \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \rightarrow F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \rightarrow \mathbf{id}$
$T * F$	$T * F$	$E \rightarrow T * F$

30. Identify the tokens generated by the statement a := b * d – 60

Answer:

Suppose we pass a statement through lexical analyzer:

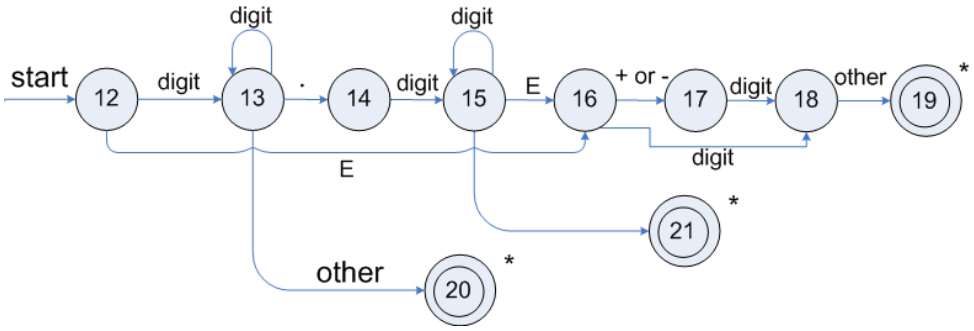
a:=b*d–60

It will generate token sequence like this:

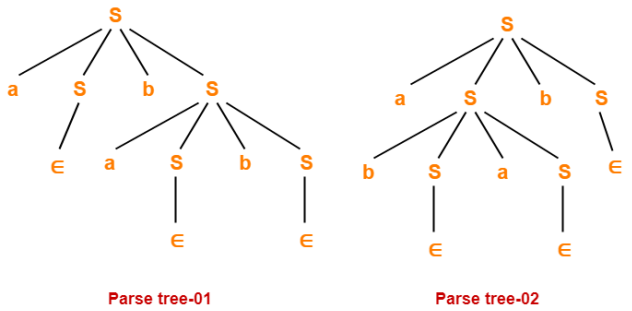
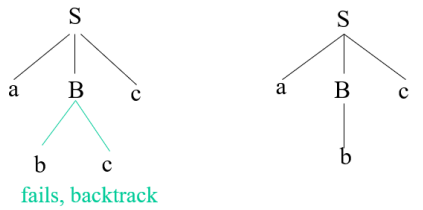
Id:=id*id-60

31. Construct the transition diagram for an unsigned constant.

Answer:



32. Show that the following grammar is Ambiguous Grammar or not.

	S → aSbS S → bSaS S → ε **Answer:** Let us consider a string w generated by the given grammar w = abab Now, let us draw parse trees for this string w.  Since two different parse trees exist for string w, therefore the given grammar is ambiguous.
33.	Illustrate that how backtracking can be used in recursive decent parser. **Answer:** • Recursive descent with backtracking is a technique that determines which production to use by trying each production in turn. • Recursive descent parsers can't be used for left-recursive grammars. • It tries to find the left-most derivation. **Example:** S → aBc B → bc | b input: abc 

PART-B

1.	Describe the term compiler and explain different phases of a compiler and show how the source code a := b + d * 60 q:=r+t*45 are translated at various phases.
2.	Explain the role of parser with an example.
3.	Consider the grammar **S → (L) | a** **L → L,S | S** a. Eliminate Left-recursion b. Find FIRST and FOLLOW c. Construct a predictive parser d. Check that the string **(a, (a,a))**accepted by given grammar.
4.	Describe the **Synthesis part of a compiler** and show how the given intermediate code is translated to object code at various phases for an example: **position=initial+rate*10**
5.	Construct the DFA for the following regular expression using direct method: **(a+b)*abb**
6.	Construct a recursive descent parser with backtracking for the following grammar:

	S->cAd **A->ab/a** for the string cad.
7.	Consider the grammar **E->E+T/T** **T->T*F/F** **F->(E)/id** a. Eliminate Left-recursion b. Find First and Follow c. Construct a predictive parser d. Show the behavior of the parser on the string **id+id*id**
8.	Construct Stack implementation of shift reduce parsing for the grammar and parse the input string id1+id2*id3 **E->E+E** **E->E*E** **E->(E)** **E->id**
9.	Construct a Deterministic Finite Automata recognizing the language containing string that are multiples of 4 when represented in binary.
10.	Inspect the role of Lexical analyzer with suitable examples.
11.	Write the algorithm for the construction of LR parsing table for the given grammar. Also construct the LR parsing table for the grammar **E->E+T/T** **T->T*F/F** **F->E/id**

DEPARTMENT OF INFORMATION TECHNOLOGY

23CSC04-PRINCIPLES OF COMPILER DESIGN

QUESTION BANK MODULE- II

COURSE OUTCOMES	
C601.1	Construct a lexical analyzer to identify the tokens in a program [AP]
C601.2	Construct a parser through the application of grammar. [AP]
C601.3	Discuss the intermediate code generation and symbol table organization techniques. [U]

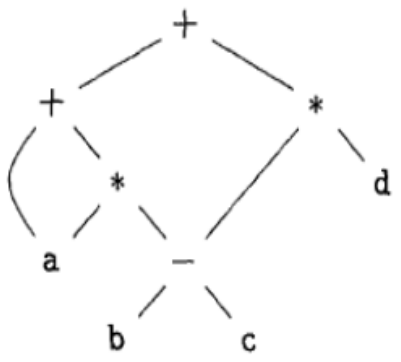
C601.4	Explain target machine's run time environment. [U]
C601.5	Construct a compiler for a small language with code generation and optimization strategies. [AP]

ANSWER ALL QUESTIONS				
	PART A (9 x 2 Marks = 18 Marks)	**BT**	**CO**	**MARKS**

1.	Construct three-address code for
	While(i < 10)
	{
	x=0;
	i=i+1;
	}
		AP	C601.3	2

2.	What do you mean by Type System?	U	C601.3	2
3.	Construct Triples for an expression: a*(b+c).	AP	C601.3	2
4.	Write the formula for calculating relative address of A[i,j], Where A is the 2- D array stored in row-major form. Let A be a 10x20 array with low 1 =low 2 =1. Therefore n 1 =10 and n 2 =20. Take w to be 4.Give the annotated parse tree and three address code for the assignment x:=A[i,j]	AP	C601.3	2
5.	Illustrate syntax directed definition for if-else statement.	AP	C601.3	2
6.	State heap allocation strategy.	U	C601.4	2
7.	Annotate the term coercion with an example.	U	C601.3	2
8.	Discover the significance of Back Patching.	U	C601.3	2
9.	Compare Storage organization techniques.	U	C601.4	2
10	List the different types of three address statements.	R	C601.3	2
11	Differentiate between syntax tree and parse tree.	U	C601.3	2
12	Construct Quadruples for an expression: a = b * - c + b * -c	AP	C601.3	2
13	Write the semantic rules for E->id_1 relop id_2 using Backpatching.	AP	C601.3	2
14	Consider the following grammar:	AP	C601.3	2

	E → E + T | T T → T x F | F F → (E) | id Derive the string **id + id x id** using Directed Acyclic Graph (DAG)			
15	Annotate the term coercion with an example.	U	C601.3	2
16	Indicate the components present in activation record.	U	C601.4	2
17	Draw the activation tree for Quick sort.	AP	C601.4	2
18	Compare Storage organization techniques.	U	C601.4	2
19	List out various ways of representing intermediate languages.	R	C601.3	2
20	Distinguish between triples and indirect triples.	A	C601.3	2
21	Build the following code in Quadruples T1:=A*B T2=R/S X=T1+T2	AP	C601.3	2
22	Translate the arithmetic expressions a*-(b+c) into postfix notation.	U	C601.2	2
23	List Storage allocation strategies.	R	C601.4	2
24	Infer three address code sequence for the assignment statement d:=(a-b)+(a-c)+(a-c).	U	C601.3	2
25	List the source language issues.	R	C601.4	2
26	Construct DAG for the expression a+a*(b-c) +(b-c) *d	AP	C601.3	2
27	Define backpatching.	R	C601.3	2
28	Construct a syntax tree for the expression a=b*-c+b*-c	R	C601.3	2
29	Distinguish between Quadruple, Triple and Indirect triple.	A	C601.3	2
30	Build the following code in Quadruples T1:=A*B T2=R/S X=T1+T2	AP	C601.3	2
31	List the differences between S-Attribute and L-Attribute.	U	C601.2	2
32	Write the three address for given DAG.	R	C601.4	2

				
33	Infer three address code sequence for the assignment statement d:=(a-b)+(a-c)+(a-c).	U	C601.3	2
34	Define: Dependency graph with example.	R	C601.4	2
35	Discuss the intermediate code representation for “a or b not c”	AP	C601.3	2
36	Describe the Storage allocation strategies.	R	C601.3	2

		PART B	**BT**	**CO**	**MARKS**
1.	i.	Discuss the translation scheme to produce three-address code for assignments and compute the three-address code for the following statement. a=(b+-c)*d	AP	C601.3	8
	ii.	Illustrate different Storage Allocation Strategies used to store variables during run time environment with neat diagram.	AP	C601.4	8
2	i.	Explain the translation scheme to produce three-address code for the following control flow statements a. S- >while E do S1 b. S- >if E then S1 else S2 Construct Syntax tree and annotated parse tree for the below	AP	C601.3	16

		source code using SDD.
		While a <b do
		if c < d then
		x=y+z
		else
		x=y-z			

3	i)	Explain the specification of simple type checker for expressions and statements.	U	C601.3	8
4	i)	Explain Backpatching and generate the three address code and syntax tree for the following expression a<b or c<d and e<f using backpatching.	AP	C601.3	8
5	i)	Construct a syntax directed definition for constructing a syntax tree for assignment statements. S->id:=E E->E1+E2 E->E1*E2 E->-E1 E->(E1) E->id	AP	C601.3	8
	ii)	Illustrate type checking with necessary diagram.	U	C601.3	8
6	i)	Explain about runtime storage management.	U	C601.4	16
7	i)	Construct a syntax directed definition for constructing a syntax tree for assignment statements. S->id:=E E->E1+E2 E->E1*E2 E->-E1 E->(E1) E->id	AP	C601.3	8
	ii)	Discuss in detail about Intermediate code for Switch statement.	U	C601.3	8
8	i)	Illustrate three address code and its types with examples.	U	C601.4	8
	ii)	Using Syntax directed translation, Convert the expression 3+5*6 into postfix expression. E□ E+T { print (“+”);} E□ T { } T□T * F { print (“ * “);} T□ F { } F□ num { print (“ num= lexval”);}	AP	C601.3	8

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution and affiliated to Anna University, Chennai)
Kuniamuthur, Coimbatore - 641 008

23CSC04-PRINCIPLES OF COMPILER DESIGN

QUESTION BANK MODULE- III

ANSWER ALL QUESTIONS				
		BT	**CO**	**MARKS**
37	Define basic block. How will you determine the set of leaders in basic block?	R	C601.5	2

38	Calculate instruction cost for the following assembly language
	MOV *R1, *R2
	ADD #1,*R2
	SUB M,R2
		AP	C601.5	2

39	j=0
	if(j==2)
	{
	x=y+z;
	}
	What kind of optimization technique can be applied to optimize the given three address code? Justify
		AP	C601.5	2

40	Construct target code for the statement a=b+c		C601.5	2

	a=a+1	AP		
41	Recall the term flowgraph. Show that how to represent loops in a flowgraph with suitable example.	U	C601.5	2
42	Construct DAG for given three address code T1=a+b T2=T1+c T3=T1*T2	AP	C601.5	2
43	Examine the machine dependent peephole optimization techniques	U	C601.5	2
44	List out the use of register descriptor and address descriptor?	R	C601.5	2
45	Identify the advantages of data flow equations.	U	C601.5	2
46	List out the primary structure preserving transformations on basic block.	R	C601.5	2
47	Write the code sequence for d: =(a-b)+(a-c)+(a-c)	AP	C601.5	2
48	Illustrate the term loop optimization in code optimization.	U	C601.4	2
49	Write the steps to partition a sequence of three address statements into basic blocks.	AP	C601.5	2
50	State register descriptor and address descriptor.	A	C601.4	2
51	Define data flow analysis.	R	C601.5	2
52	Calculate the cost of following code sequence: i)MOV B,A ii) ADD C,A	A	C601.5	2
53	Define Just in time compiler.	R	C601.5	2
54	Illustrate the following code using flow graph. i=1;sum=0;while(i<=10){ sum +=i;i++}	AP	C601.5	2
55	Differentiate Absolute machine language and Re-locatable machine language in code generation.	R	C601.5	2
56	Discuss the Register allocation and Register assignment problems arise when we use registers.	U	C601.5	2

57	Find the total instruction cost for the given code:
	MOV R0, R1
	MOV R0, M
	MOV #1, R0
	MOV 4(R0), *12(R1)
		AP	C601.4	2

58	List out the steps in Leader selection process in Basic block construction.	U	C601.5	2
59	Check the given code is part of single basic block. Justify your answer. (1) If X<Y GOTO (4) (2) T1=0 (3) GOTO (5) T1=1	A	C601.4	2
60	Define: Data flow analysis.	R	C601.5	2
61	Draw Direct acyclic graph for the given expression (((a+a)+(a+a))+ ((a+a)+(a+a)))	AP	C601.5	2
62	Discuss the objectives in Peephole optimization.	R	C601.5	2

63	Illustrate the following code using flow graph.
	I=1;
	SUM=0;
	WHILE(I<=10)
	{
	SUM +=I;
	I++
	}
		AP	C601.5	2

64	List the issues in the design of code generator.	R	C601.5	2
65	Develop the three address code for **a+b-(c+d)*e** and apply the choice of evaluation order for the obtained code.	AP	C601.5	2
66	Manipulate the Instruction Cost for the three address statement i. MOV R0,R1 ii. MOV R0,M iii. ADD #1,R0 iv. SUB 2(R0),*8(R1)	AP	C601.5	2
67	Illustrate the DAG for the expression below: a=b+c, b=b-d, c=c+d, e=b+c	AP	C601.5	2
68	Discover the properties of optimizing compilers.	U	C601.5	2
69	Recite the techniques used for loop optimization.	R	C601.5	2
70	Differentiate Local transformation & Global Transformation.	U	C601.5	2
71	Examine the use of peephole optimization.	R	C601.5	2
72	Label the primary structure preserving transformations on basic blocks.	R	C601.5	2

PART B Questions can be either one division with 16 Marks /(8+8) Marks/10+6 Marks		
1.	i)	Explain structure preserving transformation and Algebraic transformation techniques with suitable Example.
2.	i)	Examine the simple code generation algorithm by constructing the code for the following statement: d=(a-b)+(a-c)+(a-c)
3.		Consider the following code begin p=0 i=1 do begin p=p+a[i]*b[i] i=i+1 end while i<=20 j=j+1 end 1.Translate the program into three address statements. Assume that size of each number in array requires 4 bytes. 2.Construct Flow graph for the code from 1 3. Identify the loops in the flow graph from 2.
4		Explain in detail about principle sources of optimization.
5		Describe in detail about optimization of basic blocks with example.

6		Illustrate DAG representation of the following code and list out the applications of DAG representation:
		
		i=1;s=0;
		
		while(i<=10)
		
		s=s+a[i][i];
		
		i=i+1;

7		Explain simple code generator with suitable example.
		Illustrate the below Intermediate code to set a 10*10 matrix to an identity matrix as basic blocks and its corresponding flow graph. 1) i = 1 2) j = 1 3) t1 = 10 * i 4) t2 = t1 + j 5) t3 = 8 * t2 6) t4 = t3 - 88 7) a[t4] = 0.0 8) j = j + 1 9) if j <= 10 goto (3) 10) i = i + 1 11) if i <= 10 goto (2) 12) i = 1 13) t5 = i - 1 14) t6 = 88 * t5 15) a[t6] = 1.0 16) i = i + 1 17) if i <= 10 goto (13)
8		Dramatize the code generator algorithm and cconstruct the Register descriptor and address descriptor for the statement D=(a-b)+(a-c)+(a-c) using the algorithm.
9		Explain in brief about different Principal sources of optimization techniques with suitable examples.
10		Discuss how the optimization is carried out in Peephole optimization technique. Also, List the advantages.
11		Draw Direct acyclic graph for the following three address code: (1) T1:= 4*I (2) T2:= A[T1] (3) T3:=4*I (4) T4:= B[T3] (5) T5:=T2 * T4 (6) T6:= PROD + T5 (7) PROD:= T6 (8) T7:= I+1 (9) I:=T7 IF I<= 20 GOTO (1)
12		Find the basic blocks and flow graphs for the given code, Explain the steps in detail. (1) A=1 (2) T1=5 (3) T2=A*3 (4) B=T1+T2 (5) R=5-T1 (6) IF A>R GOTO (3) (7) B=T1+A E=5+B

13		Discuss about the principle sources of optimization with necessary examples each.